

P1154R0

Type traits for structural comparison

Published Proposal, 2018-10-04

Authors:

[Arthur O'Dwyer](#)

[Jeff Snyder](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Source:

github.com/Quuxplusone/draft/blob/gh-pages/d1154-comparable-traits.bs

Current:

rawgit.com/Quuxplusone/draft/gh-pages/d1154-comparable-traits.html

Abstract

Following on the heels of [P0732 "Class Types in Non-Type Template Parameters"](#), we propose the new type-trait `has_strong_structural_equality<T>`. This trait improves code maintainability by letting the programmer `static_assert` an important property of their class types.

Table of Contents

- 1 Strong structural equality should be static-assertable
- 2 This feature requires support from the compiler
- 3 Provide a full complement of type traits
- 4 Proposed wording

References

Informative References

§ 1. Strong structural equality should be static-assertable

The concept of "having strong structural equality" that was introduced by [\[P0732\]](#) will become important to programmers. Take this class type for example:

```
template<class CharT, std::size_t N>
struct basic_fixed_string
{
    constexpr basic_fixed_string(const CharT (&foo)[N+1])
        { std::copy_n(foo, N+1, m_data); }
    friend auto operator <=> (const basic_fixed_string &,
                             const basic_fixed_string &) = default;

    CharT m_data[N+1];
};
```

This type's `operator<=>` is a "structural comparison operator" because it is defaulted, and invokes only other structural comparison operators. Since it yields `strong_ordering`, it also has what we might call "strong structural ordering", although this term is not introduced by P0732. "Strong structural ordering" is a stricter requirement than P0732's "strong structural equality," which is the property required for using values of user-defined type as non-type template parameters.

We propose that C++ should permit the programmer to test the presence or absence of this property. Example:

```
static_assert(std::has_strong_structural_equality_v< basic_fixed_string<5>
```

This permits maintainability-minded programmers to express their intention in code.

```

template<class CharT, std::size_t N>
struct broken_fixed_string
{
    constexpr broken_fixed_string(const CharT (&foo)[N+1])
        { std::copy_n(foo, N+1, m_data); }
    friend auto operator <=> (const broken_fixed_string& a,
                             const broken_fixed_string& b)
        { return std::memcmp(a.m_data, b.m_data, N+1) <=> 0; }
    CharT m_data[N+1];
};
static_assert(std::has_strong_structural_equality_v<broken_fixed_string<5>>
    "broken_fixed_string lacks the strong structural equality we expected")

// ... possibly many lines of code here ...

template<auto V> struct A {};
A<broken_fixed_string("hello")> a;

```

In the snippet above, we get a nice descriptive `static_assert` failure, instead of an unfriendly spew of diagnostics on the line that tries to instantiate `A`.

§ 2. This feature requires support from the compiler

The programmer cannot implement this type-trait in standard C++2a. They can get very close:

```

template<auto V> constexpr int Test() { return 0; }

template<class T, class = int>
struct HasStrongStructuralEquality : std::false_type {};

template<class T>
struct HasStrongStructuralEquality<T, decltype(Test<T{}>())> : std::true_type {};

static_assert(HasStrongStructuralEquality<int>::value);
static_assert(!HasStrongStructuralEquality<std::string>::value);

```

But this approach falters when `T` is not constexpr default-constructible. We have seen this obstacle before; it produced the now-deprecated type-trait `is_literal_type<T>`, which likewise cannot be

implemented in standard C++. The present situation is exactly analogous.

§ 3. Provide a full complement of type traits

We propose the following type-traits, with their accompanying `_v` versions. For exposition purposes only, we provide sample implementations in terms of a hypothetical GCC/Clang builtin

`__has_structural_comparison(T)`.

```
template<class T> struct has_structural_comparison :
    bool_constant< __has_structural_comparison(T) > {};

template<class T> struct has_strong_structural_ordering :
    bool_constant<
        __has_structural_comparison(T) &&
        is_convertible_v<decltype(declval<T>()) <= declval<T>()), strong_ordering>
    > {};

template<class T> struct has_strong_structural_equality :
    bool_constant<
        __has_structural_comparison(T) &&
        is_convertible_v<decltype(declval<T>()) <= declval<T>()), strong_equality>
    > {};

template<class T> struct has_weak_structural_ordering :
    bool_constant<
        __has_structural_comparison(T) &&
        is_convertible_v<decltype(declval<T>()) <= declval<T>()), weak_ordering>
    > {};

template<class T> struct has_weak_structural_equality :
    bool_constant<
        __has_structural_comparison(T) &&
        is_convertible_v<decltype(declval<T>()) <= declval<T>()), weak_equality>
    > {};

template<class T> struct has_partial_structural_ordering :
    bool_constant<
        __has_structural_comparison(T) &&
        is_convertible_v<decltype(declval<T>()) <= declval<T>()), partial_ordering>
    > {};
```

§ 4. Proposed wording

Add new entries to Table 46 in [\[meta.unary.prop\]](#):

Template	Condition	Preconditions
<code>template<class T> struct has_structural_comparison;</code>	For a glvalue <code>x</code> of type <code>const T</code> , the expression <code>x <=> x</code> either does not invoke a three-way comparison operator or invokes a structural comparison operator (15.9.1).	T shall be a complete type, cv <code>void</code> , or an array of unknown bound.
<code>template<class T> struct has_strong_structural_ordering;</code>	<code>has_structural_comparison_v<T></code> is true and the expression <code>x <=> x</code> is convertible to <code>std::strong_ordering</code> .	T shall be a complete type, cv <code>void</code> , or an array of unknown bound.
<code>template<class T> struct has_strong_structural_equality;</code>	<code>has_structural_comparison_v<T></code> is true and the expression <code>x <=> x</code> is convertible to <code>std::strong_equality</code> .	T shall be a complete type, cv <code>void</code> , or an array of unknown bound.
<code>template<class T> struct has_weak_structural_ordering;</code>	<code>has_structural_comparison_v<T></code> is true and the expression <code>x <=> x</code> is convertible to <code>std::weak_ordering</code> .	T shall be a complete type, cv <code>void</code> , or an array of unknown bound.
<code>template<class T> struct has_weak_structural_equality;</code>	<code>has_structural_comparison_v<T></code> is true and the expression <code>x <=> x</code> is convertible to <code>std::weak_equality</code> .	T shall be a complete type, cv <code>void</code> , or an array of unknown bound.
<code>template<class T> struct has_strong_structural_ordering;</code>	<code>has_structural_comparison_v<T></code> is true and the expression <code>x <=> x</code> is convertible to <code>std::strong_ordering</code> .	T shall be a complete type, cv <code>void</code> , or an array of unknown bound.
<code>template<class T> struct has_partial_structural_ordering;</code>	<code>has_structural_comparison_v<T></code> is true and the expression <code>x <=> x</code> is convertible to <code>std::partial_ordering</code> .	T shall be a complete type, cv <code>void</code> , or an array of unknown bound.

§ References

§ Informative References

[P0732]

Jeff Snyder; Louis Dionne. [Class Types in Non-Type Template Parameters](http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0732r2.pdf). June 2018. URL: <http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2018/p0732r2.pdf>

[SO]

plasmacel; Nicol Bolas. [Deprecated std::is_literal_type in C++17](https://stackoverflow.com/questions/40351816/deprecated-stdis-literal-type-in-c17). October 2016. URL: <https://stackoverflow.com/questions/40351816/deprecated-stdis-literal-type-in-c17>